REVA University

Department of Electrical & Electronics Engineering

TOPICE: Statistical Analysis of Body Measurements using Python and NumPy

A Data Science Project

Dataset Used: National Health and Nutrition Examination Survey (NHANES) 2020
(Source: github.com/gagolews/teaching-data)

Tools & Technologies: Python | NumPy | Matplotlib | Seaborn | SciPy | Google Colab

Submitted By:

Name	: M. Sri Hari Datta
USN	: R23EM049
Semester	: 6th Sem
Branch	: B.E — EEE

CELL 1→

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from scipy import stats

print("All libraries loaded!")
```

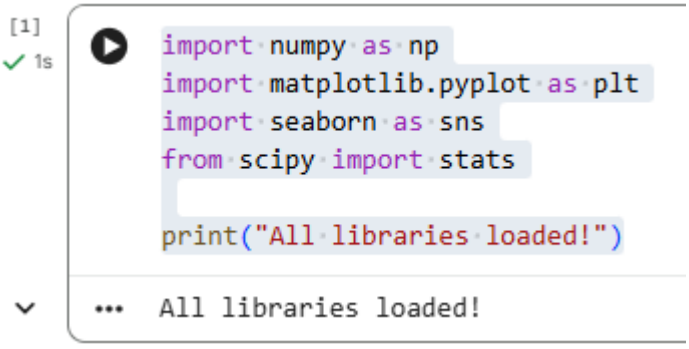
import numpy as np — for matrix operations, shortcut np

import matplotlib.pyplot as plt — for plotting graphs, shortcut plt

import seaborn as sns — for pairplot, shortcut sns

from scipy import stats — for skewness, kurtosis, correlation

print("All libraries loaded!") — confirms no import errors



All libraries loaded! — all 4 libraries imported successfully, no errors. ready to start the project.

Cell 2 →

```
import urllib.request
```

```
urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/gagolews/teaching-
data/master/marek/nhanes_adult_male_bmx_2020.csv",
    "nhanes_adult_male_bmx_2020.csv"
)

urllib.request.urlretrieve(
    "https://raw.githubusercontent.com/gagolews/teaching-
data/master/marek/nhanes_adult_female_bmx_2020.csv",
    "nhanes_adult_female_bmx_2020.csv"
)

print("Files downloaded!")
```

import urllib.request — imports library used to download files from internet

urllib.request.urlretrieve(...male...csv) — downloads male dataset from github and saves it locally as nhanes_adult_male_bmx_2020.csv

urllib.request.urlretrieve(...female...csv) — same but for female dataset

print("Files downloaded!") — confirms both files downloaded successfully without errors

Files downloaded! — both male and female csv files successfully downloaded from github and saved in colab. no errors occurred.

Cell 3→

```
male = np.genfromtxt("nhanes_adult_male_bmx_2020.csv", delimiter=",",
skip_header=1)
female = np.genfromtxt("nhanes_adult_female_bmx_2020.csv", delimiter=",",
skip_header=1)

male = male[~np.isnan(male).any(axis=1)]
female = female[~np.isnan(female).any(axis=1)]

print("Male shape:", male.shape)
print("Female shape:", female.shape)
print("Male first row:", male[0])
print("Female first row:", female[0])
```

male = np.genfromtxt(...) — reads male csv file and stores it as numpy matrix. delimiter="," means comma separated values. skip_header=1 skips the first title row.

female = np.genfromtxt(...) — same but for female dataset.

male = male[~np.isnan(male).any(axis=1)] — removes any rows that have NaN (missing) values from male matrix.

female = female[~np.isnan(female).any(axis=1)] — same for female matrix.

print("Male shape:", male.shape) — prints how many rows and columns male matrix has.

print("Female shape:", female.shape) — same for female.

print("Male first row:", male[0]) — prints first row of male matrix to verify data loaded correctly.

print("Female first row:", female[0]) — same for female.

[3]
✓ 0s

```
male = np.genfromtxt("nhanes_adult_male_bmx_2020.csv", delimiter=",", skip_header=1)
female = np.genfromtxt("nhanes_adult_female_bmx_2020.csv", delimiter=",", skip_header=1)

male = male[~np.isnan(male).any(axis=1)]
female = female[~np.isnan(female).any(axis=1)]

print("Male shape:", male.shape)
print("Female shape:", female.shape)
print("Male first row:", male[0])
print("Female first row:", female[0])
```

```
Male shape: (4081, 7)
Female shape: (4221, 7)
Male first row: [ 98.8 182.3  42.   40.1  38.2 108.2 120.4]
Female first row: [ 97.1 160.2  34.7  40.8  35.8 126.1 117.9]
```

Male shape: (4081, 7) — male matrix has 4081 persons and 7 body measurements.

Female shape: (4221, 7) — female matrix has 4221 persons and 7 body measurements.

Male first row: [98.8 182.3 42. 40.1 38.2 108.2 120.4] — first male person weighs 98.8kg, height 182.3cm, and so on for remaining 5 measurements.

Female first row: [97.1 160.2 34.7 40.8 35.8 126.1 117.9] — first female person weighs 97.1kg, height 160.2cm, and so on. data loaded correctly, no NaN values.

Cell 4→.

```
male_weights = male[:, 0]
female_weights = female[:, 0]

x_min = min(male_weights.min(), female_weights.min()) - 5
x_max = max(male_weights.max(), female_weights.max()) + 5

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(female_weights, bins=30, color='hotpink', edgecolor='black')
plt.xlim(x_min, x_max)
plt.title("Female Weights")
plt.xlabel("Weight (kg)")
plt.ylabel("Frequency")

plt.subplot(1, 2, 2)
plt.hist(male_weights, bins=30, color='steelblue', edgecolor='black')
plt.xlim(x_min, x_max)
plt.title("Male Weights")
plt.xlabel("Weight (kg)")
plt.ylabel("Frequency")

plt.suptitle("Distribution of Weights: Female vs Male", fontsize=14)
plt.tight_layout()
plt.show()
```

male_weights = male[:, 0] — extracting column 0 (weight) from male matrix. [:, 0] means all rows, column 0.

female_weights = female[:, 0] — same for female matrix.

x_min = min(...) - 5 — finding lowest weight from both male and female, subtracting 5 for extra space on left of graph.

x_max = max(...) + 5 — finding highest weight from both, adding 5 for extra space on right.

plt.figure(figsize=(12, 5)) — creating blank figure of size 12x5 inches.

plt.subplot(1, 2, 1) — dividing figure into 1 row 2 columns, selecting position 1 for female histogram.

plt.hist(female_weights, bins=30, color='hotpink') — drawing female histogram with 30 bars in pink color.

plt.xlim(x_min, x_max) — setting same x axis limits for both graphs so comparison is fair.

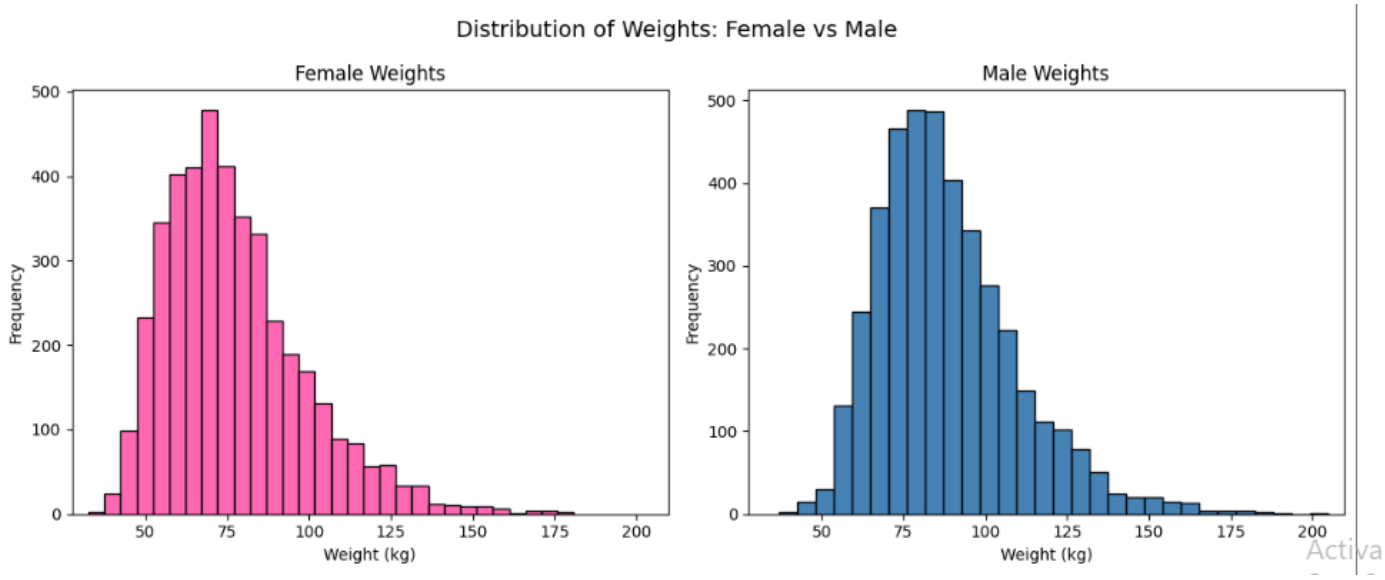
plt.subplot(1, 2, 2) — selecting position 2 for male histogram.

plt.hist(male_weights, bins=30, color='steelblue') — drawing male histogram in blue with 30 bars.

plt.suptitle(...) — adding one common title for both graphs.

plt.tight_layout() — adjusts spacing so graphs don't overlap.

plt.show() — displays the final figure.



both histograms plotted correctly with same x axis range 40 to 200kg.

female histogram — most females weigh between 60-80kg. graph is right skewed meaning few females are very heavy above 125kg.

male histogram — most males weigh between 75-95kg. also right skewed but peak is shifted more to the right meaning males are heavier overall.

comparison — both distributions have same x axis so comparison is fair. males are clearly heavier than females on average.

Cell 5→

```
plt.figure(figsize=(8, 6))

plt.boxplot([female_weights, male_weights],
            labels=['Female', 'Male'],
            patch_artist=True,
            boxprops=dict(facecolor='hotpink'),
            medianprops=dict(color='black', linewidth=2))

plt.title("Box-and-Whisker Plot: Female vs Male Weights")
plt.ylabel("Weight (kg)")
plt.grid(True, alpha=0.3)
plt.show()
```

`plt.figure(figsize=(8, 6))` — creating blank figure of size 8x6 inches.

`plt.boxplot([female_weights, male_weights],...)` — drawing two boxes side by side, female first then male.

`labels=['Female', 'Male']` — giving names to both boxes.

`patch_artist=True` — allows filling color inside the boxes.

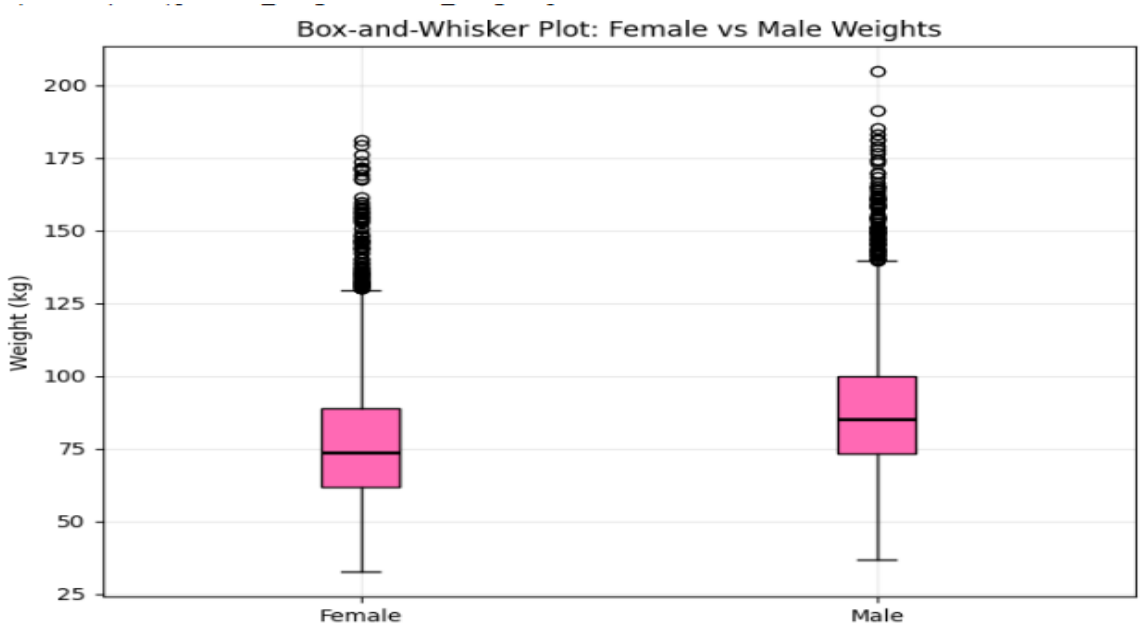
`boxprops=dict(facecolor='hotpink')` — filling boxes with pink color.

`medianprops=dict(color='black', linewidth=2)` — median line inside box set to black with thickness 2.

`plt.ylabel("Weight (kg)")` — labeling y axis as Weight in kg.

`plt.grid(True, alpha=0.3)` — adding light grid in background, 30% visible.

`plt.show()` — displays the final boxplot.



female box — median is around 73kg. box range is 63 to 88kg. many outliers above 130kg meaning few very heavy females.

male box — median is around 85kg. box range is 73 to 97kg. males are clearly heavier than females overall.

comparison — male box is higher than female box confirming males weigh more on average. both have lots of outlier dots on top meaning both groups have some very heavy people. male box is slightly bigger meaning male weights are more spread out.

Cell 7→

```
print("=" * 45)
print("        FEMALE WEIGHT STATISTICS")
print("=" * 45)
print(f"Mean        : {np.mean(female_weights):.2f} kg")
print(f"Median      : {np.median(female_weights):.2f} kg")
print(f"Std Dev     : {np.std(female_weights):.2f} kg")
print(f"Variance    : {np.var(female_weights):.2f}")
print(f"Min         : {np.min(female_weights):.2f} kg")
print(f"Max         : {np.max(female_weights):.2f} kg")
print(f"Skewness    : {stats.skew(female_weights):.4f}")
print(f"Kurtosis    : {stats.kurtosis(female_weights):.4f}")

print("\n" + "=" * 45)
print("         MALE WEIGHT STATISTICS")
print("=" * 45)
print(f"Mean        : {np.mean(male_weights):.2f} kg")
print(f"Median      : {np.median(male_weights):.2f} kg")
print(f"Std Dev     : {np.std(male_weights):.2f} kg")
print(f"Variance    : {np.var(male_weights):.2f}")
print(f"Min         : {np.min(male_weights):.2f} kg")
print(f"Max         : {np.max(male_weights):.2f} kg")
print(f"Skewness    : {stats.skew(male_weights):.4f}")
print(f"Kurtosis    : {stats.kurtosis(male_weights):.4f}")
```

print("=" * 45) — prints a line of 45 equal signs just for visual separation.

print("FEMALE WEIGHT STATISTICS") — prints heading for female section.

np.mean(female_weights) — calculates average weight of all females.

np.median(female_weights) — calculates middle value of female weights.

np.std(female_weights) — calculates standard deviation, how much weights vary from average.

np.var(female_weights) — calculates variance, square of standard deviation.

np.min(female_weights) — finds lowest female weight.

np.max(female_weights) — finds highest female weight.

stats.skew(female_weights) — calculates skewness, tells if distribution is left or right skewed.

stats.kurtosis(female_weights) — calculates kurtosis, tells how many extreme outliers are present.

same functions repeated below for male weights.

:.2f — used in all print statements to round values to 2 decimal places.

```
...  ============================================
             FEMALE WEIGHT STATISTICS
     ============================================
     Mean        : 77.40 kg
     Median      : 73.60 kg
     Std Dev     : 21.54 kg
     Variance    : 464.08
     Min         : 32.60 kg
     Max         : 180.90 kg
     Skewness    : 1.0334
     Kurtosis    : 1.4010

     ============================================
              MALE WEIGHT STATISTICS
     ============================================
     Mean        : 88.36 kg
     Median      : 85.00 kg
     Std Dev     : 21.42 kg
     Variance    : 458.77
     Min         : 36.80 kg
     Max         : 204.60 kg
     Skewness    : 0.9843
     Kurtosis    : 1.4743
```

Female:

- mean 77.40kg, median 73.60kg — mean is higher than median so distribution is right skewed
- std dev 21.54 means weights vary by 21kg from average
- min 32.60kg, max 180.90kg — wide range
- skewness 1.03 confirms right skewed, most females are lighter with few very heavy ones
- kurtosis 1.40 means more outliers than normal distribution

Male:

- mean 88.36kg, median 85.00kg — males are heavier than females by around 11kg
- std dev 21.42 almost same spread as females
- min 36.80kg, max 204.60kg — male max is much higher than female
- skewness 0.98 also right skewed same as females
- kurtosis 1.47 slightly more outliers than females

overall comparison:

- males are heavier than females on average
- both are right skewed meaning most people are on lighter side
- dispersion is almost same for both groups
- male has more extreme outliers on higher end

CELL 8→

```
import pandas as pd

female_bmi = female[:, 0] / ((female[:, 1] / 100) ** 2)

zfemale = (female - female.mean(axis=0)) / female.std(axis=0)

print("zfemale shape:", zfemale.shape)
print("Mean of each column:", zfemale.mean(axis=0).round(4))
print("Std of each column :", zfemale.std(axis=0).round(4))
```

```
zfemale shape: (4221, 7)
Mean of each column: [ 0.  0. -0. -0.  0. -0.  0.]
Std of each column : [1. 1. 1. 1. 1. 1. 1.]
```

```
zheight = zfemale[:, 1]
zweight = zfemale[:, 0]
zhip    = zfemale[:, 6]
zbmi    = (female_bmi - female_bmi.mean()) / female_bmi.std()

df = pd.DataFrame({
    'Height': zheight,
    'Weight': zweight,
    'Hip Circumference': zhip,
    'BMI': zbmi
})

sns.pairplot(df, plot_kws={'alpha': 0.3, 'color': 'hotpink'})
plt.suptitle("Pairplot: Height, Weight, Hip, BMI (Females)", y=1.02)
plt.show()
```

import pandas as pd — importing pandas library to create a dataframe for pairplot.

female_bmi = female[:, 0] / ((female[:, 1] / 100) ** 2) — calculating BMI for all females. formula is weight divided by height in meters squared.

zfemale = (female - female.mean(axis=0)) / female.std(axis=0) — standardizing all columns of female matrix using z-score formula. subtracting mean and dividing by std for each column.

zfemale.mean(axis=0).round(4) — checking mean of each column, should be 0 after standardization.

zfemale.std(axis=0).round(4) — checking std of each column, should be 1 after standardization.

zheight = zfemale[:, 1] — extracting standardized height column.

zweight = zfemale[:, 0] — extracting standardized weight column.

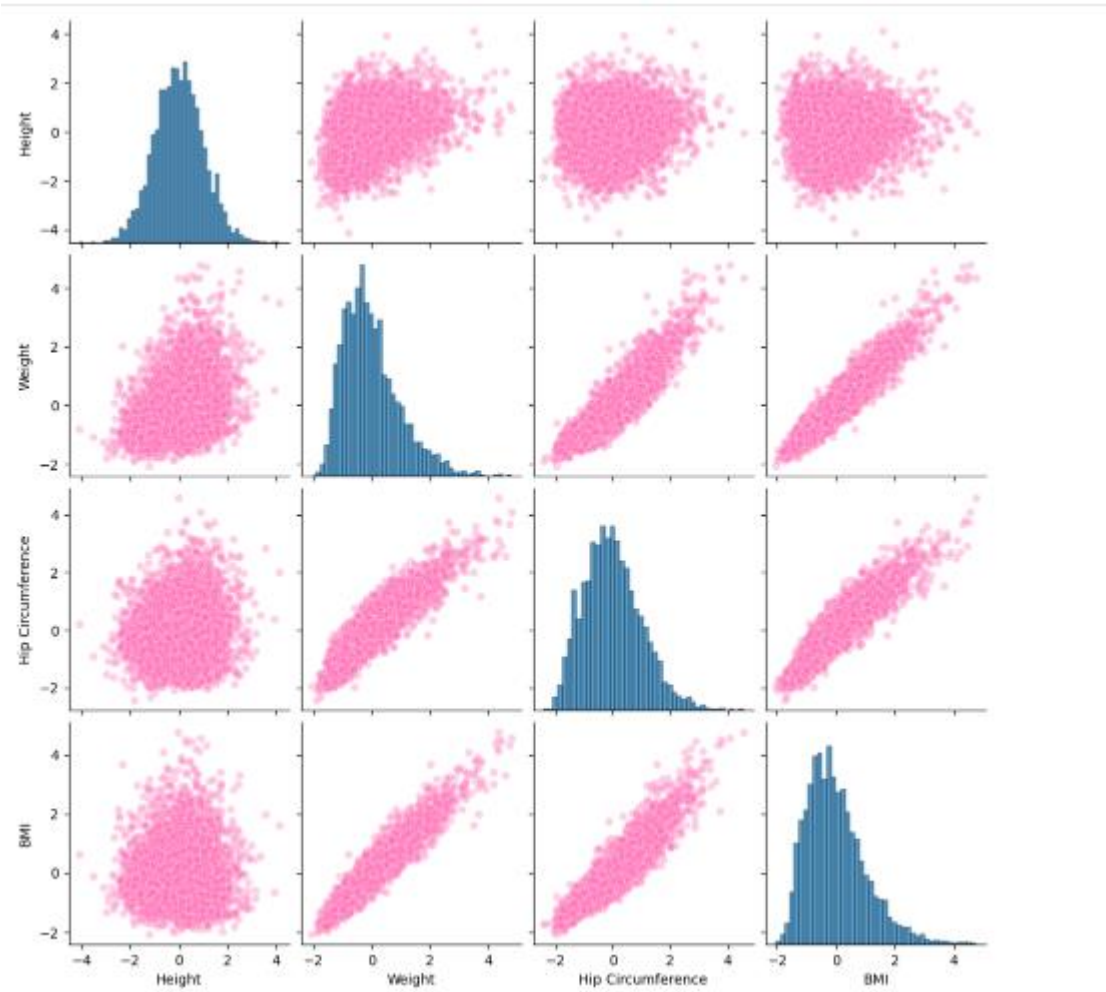
zhip = zfemale[:, 6] — extracting standardized hip circumference column.

zbmi = (female_bmi - female_bmi.mean()) / female_bmi.std() — standardizing bmi values using z-score.

df = pd.DataFrame({...}) — creating a dataframe with all 4 standardized columns for pairplot.

sns.pairplot(df,...) — drawing scatterplot matrix showing relationship between all 4 variables.

plt.suptitle(...) — adding title to the pairplot.



diagonal — histograms of each variable. height is normally distributed, weight, hip and bmi are all right skewed.

height vs weight — scattered randomly, weak positive relation between height and weight.

height vs hip — very scattered, almost no relation between height and hip size.

height vs bmi — very scattered, height has almost no effect on bmi.

weight vs hip — strong positive relation, as weight increases hip size also increases clearly.

weight vs bmi — very strong positive relation, points form a clear straight line because bmi is calculated from weight.

hip vs bmi — strong positive relation, larger hip size means higher bmi.

overall conclusion — weight, hip and bmi are all strongly related to each other. height has very little relation with any of them.

Cell 8→

```
cols = {'Height': zheight, 'Weight': zweight, 'Hip': zhip, 'BMI': zbmi}
col_names = list(cols.keys())
col_vals  = list(cols.values())

print("PEARSON CORRELATION:")
print(f"{'':12}", end="")
for n in col_names:
    print(f"{n:12}", end="")
print()
for i, n1 in enumerate(col_names):
    print(f"{n1:12}", end="")
    for j, n2 in enumerate(col_names):
        r, _ = stats.pearsonr(col_vals[i], col_vals[j])
        print(f"{r:12.4f}", end="")
    print()

print("\nSPEARMAN CORRELATION:")
print(f"{'':12}", end="")
for n in col_names:
    print(f"{n:12}", end="")
print()
for i, n1 in enumerate(col_names):
    print(f"{n1:12}", end="")
    for j, n2 in enumerate(col_names):
        r, _ = stats.spearmanr(col_vals[i], col_vals[j])
        print(f"{r:12.4f}", end="")
    print()
```

cols = {'Height': zheight, 'Weight': zweight, 'Hip': zhip, 'BMI': zbmi} — storing all 4 standardized columns in a dictionary for easy looping.

col_names = list(cols.keys()) — getting column names as a list.

col_vals = list(cols.values()) — getting column values as a list.

for i, n1 in enumerate(col_names) — looping through each column to compare with every other column.

stats.pearsonr(col_vals[i], col_vals[j]) — calculating pearson correlation between every pair of columns. value ranges from -1 to 1. closer to 1 means strong positive relation, closer to -1 means strong negative relation, closer to 0 means no relation.

stats.spearmanr(col_vals[i], col_vals[j]) — calculating spearman correlation same way. spearman is more reliable when data has outliers unlike pearson.

print(f"{r:12.4f}") — printing correlation value rounded to 4 decimal places in neat table format.

```
•••  PEARSON CORRELATION:
                Height      Weight      Hip         BMI
     Height          1.0000      0.3455      0.1265      0.0331
     Weight          0.3455      1.0000      0.9046      0.9459
     Hip             0.1265      0.9046      1.0000      0.9212
     BMI             0.0331      0.9459      0.9212      1.0000

     SPEARMAN CORRELATION:
                Height      Weight      Hip         BMI
     Height          1.0000      0.3389      0.1086      0.0199
     Weight          0.3389      1.0000      0.9002      0.9380
     Hip             0.1086      0.9002      1.0000      0.9231
     BMI             0.0199      0.9380      0.9231      1.0000
```

Pearson Correlation:

- **weight vs bmi = 0.9459** — very strong, bmi is directly calculated from weight so obvious
- **weight vs hip = 0.9046** — very strong, heavier females have larger hip size
- **hip vs bmi = 0.9212** — very strong positive relation
- **height vs bmi = 0.0331** — almost zero, height has no effect on bmi
- **height vs weight = 0.3455** — weak positive relation

Spearman Correlation:

- values are almost same as pearson
- **weight vs bmi = 0.9380** — still very strong
- **height vs bmi = 0.0199** — almost zero same as pearson

overall conclusion:

- weight, hip and bmi are all strongly correlated with each other
- height has almost no relation with bmi, hip or weight
- pearson and spearman values are very close meaning data is consistent and outliers are not affecting the results much

CELL 9→

```
female_whr  = female[:, 5] / female[:, 1]
female_whip = female[:, 5] / female[:, 6]

male_whr  = male[:, 5] / male[:, 1]
male_whip = male[:, 5] / male[:, 6]

female = np.column_stack([female, female_whr, female_whip])
male = np.column_stack([male, male_whr, male_whip])

print("Female shape:", female.shape)
print("Male shape:", male.shape)
```

```
Female shape: (4221, 9)
Male shape: (4081, 9)
```

```
plt.figure(figsize=(10, 6))

plt.boxplot(
    [female[:, 7], male[:, 7], female[:, 8], male[:, 8]],
    labels=['Female\nWaist-to-Height', 'Male\nWaist-to-Height',
            'Female\nWaist-to-Hip', 'Male\nWaist-to-Hip'],
    patch_artist=True,
    boxprops=dict(facecolor='lightblue'),
    medianprops=dict(color='red', linewidth=2)
)

plt.title("Waist-to-Height and Waist-to-Hip Ratios: Male vs Female")
plt.ylabel("Ratio Value")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

female_whr = female[:, 5] / female[:, 1] — calculating waist to height ratio for females. waist is column 5, height is column 1.

female_whip = female[:, 5] / female[:, 6] — calculating waist to hip ratio for females. waist is column 5, hip is column 6.

male_whr = male[:, 5] / male[:, 1] — same waist to height ratio for males.

male_whip = male[:, 5] / male[:, 6] — same waist to hip ratio for males.

female = np.column_stack([female, female_whr, female_whip]) — adding both ratio columns to female matrix. female now has 9 columns.

male = np.column_stack([male, male_whr, male_whip]) — same for male matrix.

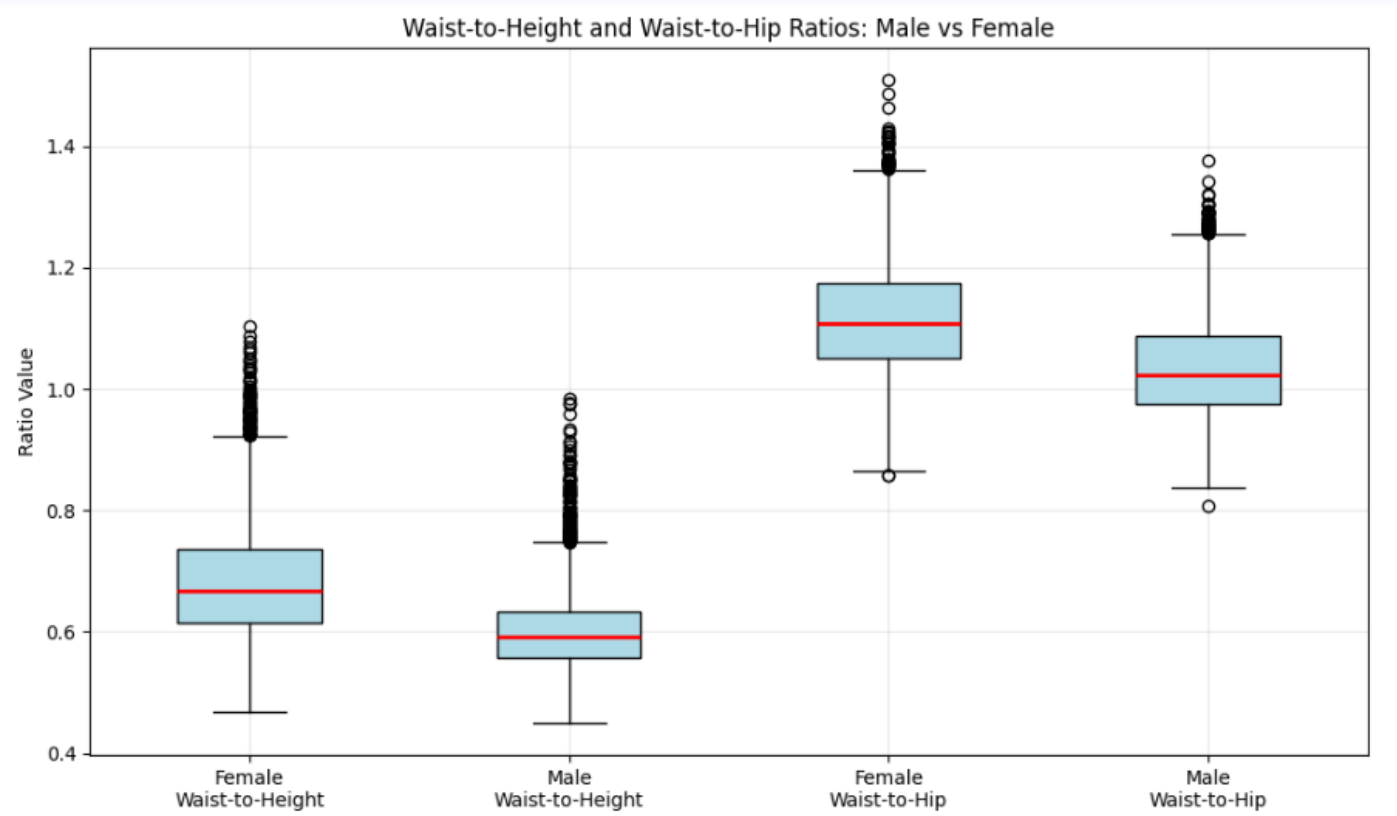
plt.boxplot([female[:, 7], male[:, 7], female[:, 8], male[:, 8]],...) — drawing 4 boxes. column 7 is waist to height ratio, column 8 is waist to hip ratio for both male and female.

patch_artist=True — allows filling color in boxes.

boxprops=dict(facecolor='lightblue') — filling boxes with light blue color.

medianprops=dict(color='red', linewidth=2) — median line set to red color.

plt.tight_layout() — adjusts spacing so labels dont overlap.



Waist-to-Height ratio:

- female median is around **0.66**, male median is around **0.60**
- females have higher waist to height ratio meaning females have larger waist relative to their height
- male box is smaller meaning less variation in males

Waist-to-Hip ratio:

- female median is around **1.10**, male median is around **1.02**
- females have higher waist to hip ratio too
- female box is bigger meaning more variation in female waist to hip ratio
- both have some outliers on higher end

overall conclusion:

- females have higher ratios in both cases compared to males
- males are taller relative to their waist size so lower waist to height ratio
- females have larger waist relative to hip size so higher waist to hip ratio
- both ratios together give better picture of obesity risk than bmi alone

CELL 10→

```
female_bmi = female[:, 0] / ((female[:, 1] / 100) ** 2)

sorted_indices = np.argsort(female_bmi)

lowest_indices  = sorted_indices[:5]
highest_indices = sorted_indices[-5:]

col_names = ["Weight", "Height", "ArmLen",
             "LegLen", "ArmCirc", "Waist",
             "Hip", "WHtR", "WHipR"]

print("=" * 75)
print("5 FEMALES WITH LOWEST BMI")
print("=" * 75)
for i, idx in enumerate(lowest_indices):
    print(f"Person {i+1}: {female[idx]} BMI={female_bmi[idx]:.2f}")

print("\n" + "=" * 75)
print("5 FEMALES WITH HIGHEST BMI")
print("=" * 75)
for i, idx in enumerate(highest_indices):
    print(f"Person {i+1}: {female[idx]} BMI={female_bmi[idx]:.2f}")
```

female_bmi = female[:, 0] / ((female[:, 1] / 100) ** 2) — calculating bmi for all females. formula is weight divided by height in meters squared.

sorted_indices = np.argsort(female_bmi) — sorting bmi values from lowest to highest and storing the row index positions.

lowest_indices = sorted_indices[:5] — taking first 5 indices which are the 5 lowest bmi females.

highest_indices = sorted_indices[-5:] — taking last 5 indices which are the 5 highest bmi females.

col_names = ["Weight", "Height", ...] — just a list of column names for reference.

for i, idx in enumerate(lowest_indices) — looping through 5 lowest bmi indices one by one.

print(f"Person {i+1}: {female[idx]} BMI={female_bmi[idx]:.2f}") — printing all 9 measurements of that person along with their bmi value.

for i, idx in enumerate(highest_indices) — same loop but for 5 highest bmi females.

```
==============================================================================
5 FEMALES WITH LOWEST BMI
==============================================================================
Person 1: [ 32.6        151.5         32.4         33.4         20.4
  77.3         64.7          0.51023102   1.19474498] BMI=14.20
Person 2: [ 36.9        158.8         32.          38.4         19.
  80.2         62.7          0.50503778   1.27910686] BMI=14.63
Person 3: [ 44.3        172.9         37.5         39.          20.
  83.          68.8          0.48004627   1.20639535] BMI=14.82
Person 4: [ 37.7        158.3         35.5         38.8         19.8
  74.          66.2          0.46746684   1.11782477] BMI=15.04
Person 5: [ 42.7        166.4         35.8         38.7         20.3
  80.6         68.7          0.484375     1.17321689] BMI=15.42

==============================================================================
5 FEMALES WITH HIGHEST BMI
==============================================================================
Person 1: [168.9        162.2         40.4         34.          46.
 173.3        164.9          1.06843403   1.05093996] BMI=64.20
Person 2: [173.4        163.7         40.          33.5         51.5
 171.4        149.           1.04703726   1.15033557] BMI=64.71
Person 3: [171.1        162.1         42.7         43.4         57.2
 170.4        163.8          1.05120296   1.04029304] BMI=65.12
Person 4: [180.9        166.4         41.          43.          53.9
 172.         169.5          1.03365385   1.01474926] BMI=65.33
Person 5: [171.4        159.9         35.9         36.6         48.3
 176.6        178.           1.10444028   0.99213483] BMI=67.04
```

5 Lowest BMI females:

- bmi range is **14.20 to 15.42** which is underweight category (below 18.5)
- weights are very low between **32.6 to 44.3 kg**
- heights are normal between **151 to 172 cm**
- waist and hip sizes are very small
- arm circumference is very small around **19-20 cm**
- these persons are clearly underweight and may have nutrition deficiency

5 Highest BMI females:

- bmi range is **64.20 to 67.04** which is extremely obese (above 40)
- weights are very high between **168.9 to 180.9 kg**
- heights are normal between **159 to 166 cm** — so high bmi is only due to high weight not height
- waist circumference is very large around **170 to 176 cm**
- waist to height ratio is above **1.0** which indicates very high risk of heart disease and diabetes
- hip circumference is also very large around **149 to 178 cm**

overall conclusion:

- bmi alone tells us if someone is underweight or obese
- but waist and hip measurements give better picture of actual health risk
- high bmi females also have very high waist ratios confirming obesity related health risks